

Kubernetes Deployment Assignment

Submitted By: Subbarayudu Satti

Batch 03 - Assignment 3 - Kubernetes

Introduction:

This assignment demonstrates deployment and management of an application using Kubernetes. The project includes containerization using Docker, deployment using Kubernetes resources, rolling updates, and health monitoring using probes.

Objective:

The main objective of this assignment is to understand:

- Pod creation
- ReplicaSet management
- Deployment management
- Rolling updates
- Kubernetes Services
- Container health checks using probes

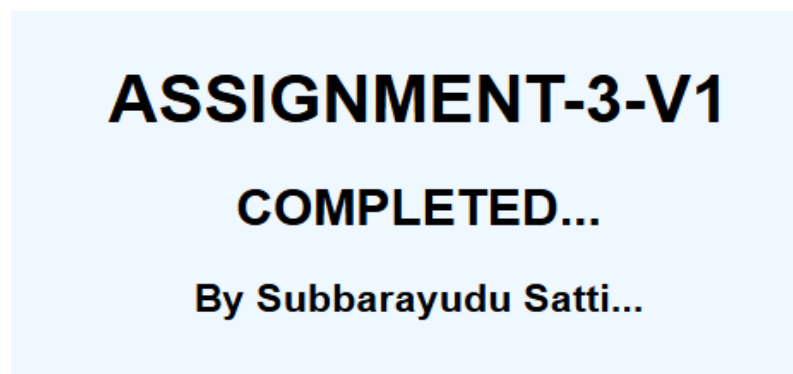
Frontend Application:

The application is a simple frontend UI created using HTML and CSS.

Version 1

Displays:

ASSIGNMENT-3-V1
COMPLETED...



Version 2

Displays:

ASSIGNMENT-3-V2
COMPLETED...

COMPLETED...

By Subbarayudu Satti...

Docker Setup:

Dockerfile:

FROM nginx:alpine

```
COPY . /usr/share/nginx/html
```

EXPOSE 80

Docker Build Commands:

Build v1

```
docker build -t subbarayudusatti/k8s-app:v1 .
```

[illegible]

Build v2

```
docker build -t subbarayudusatti/k8s-app:v2 .
```

```

$ docker build -t subbarayyagatti/k8s-appv2 .
[*] Building 1.9s (8/8) FINISHED
-> [internal] load build definition from Dockerfile
-> [transfer] GPG key ID: 1100
-> [internal] load metadata for docker.io/library/nginx:alpine
-> [auth] library/ginispinal token for registry-1.docker.io
-> [internal] load Dockerfile
-> [transfer] context: 20
-> [internal] load build context
-> [transfer] context: 330B
-> [1/2] RUN mkdir -p /usr/share/nginx/html; echo "hello world" > /usr/share/nginx/html/index.html
-> [2/2] COPY index.html /usr/share/nginx/html/index.html
-> exporting to image
-> exporting layers
-> exporting manifest sha256:b050132d0c74140ee0670137f66a20706a5047026707908012c2d3db
-> exporting config sha256:240c29d8913887353130da4d8173c32a017600331267f89e4c94d8f
-> exporting attestation manifest sha256:1216d275c9021507f620138d1ce18f5a5c4efab5a40a2924e15670eb5
-> exporting manifest list sha256:09147617161c453a300631500531f0a1fab2552d0eb0c1e4142242f1
-> naming to subbarayyagatti/k8s-appv2
-> unpacking to subbarayyagatti/k8s-appv2
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/skipcriticalbbm0w4hr0x8l

```

Docker Push Commands

docker push subbarayudusatti/k8s-app:v1

```
npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/V2 (main)
• $ docker push subbarayudusatti/k8s-app:v1
The push refers to repository [docker.io/subbarayudusatti/k8s-app]
d0ffc65fde23: Layer already exists
be5351d87f1f: Layer already exists
22ed557e9038: Layer already exists
7f32a4497b31: Layer already exists
5173742f379b: Pushed
02eda4087d06: Layer already exists
6a0ac1617861: Layer already exists
19af4ef65048: Layer already exists
b6cf024d8506: Pushed
42f24ddd8a4: Layer already exists
v1: digest: sha256:5725d2a304861710dd5e97c2327c1f259d9341dec1d42f4f47e0f3d3cc6a5ac3 size: 856

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/V2 (main)
○ $
```

docker push subbarayudusatti /k8s-app:v2

```
npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/V2 (main)
• $ docker push subbarayudusatti/k8s-app:v2
The push refers to repository [docker.io/subbarayudusatti/k8s-app]
02eda4087d06: Pushed
173d2102f8b5: Pushed
42f24ddd8a4: Pushed
22ed557e9038: Pushed
be5351d87f1f: Pushed
6a0ac1617861: Mounted from ramanachandrakiranreddy/k8s-webapp
3d7a6f2c7e8b: Pushed
19af4ef65048: Pushed
d0ffc65fde23: Pushed
7f32a4497b31: Pushed
v2: digest: sha256:7eec54c5628feca37cbb8196482e281c1a76a245f5a8dc08fe711f4fc0281f1a size: 856

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/V2 (main)
○ $
```

Kubernetes Files

pod.yml:

- Creates a single pod
- Runs container using v1 image

```
npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl apply -f pod.yaml
pod/frontend-pod created

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
frontend-pod  1/1     Running   0           8s
```

ReplicaSet (rs.yml):

- Maintains multiple pod replicas
- Ensures high availability 0

```

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl apply -f rs.yaml
replicaset.apps/frontend-rs created

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl get rs
NAME          DESIRED   CURRENT   READY   AGE
frontend-rs   3         3         3       13s

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
frontend-pod   1/1     Running   0          60s
frontend-rs-rb4tq  1/1     Running   0          19s
frontend-rs-vcnj8  1/1     Running   0          19s

```

deployment.yml

- Manages application deployment
- Supports rolling updates
- Includes probes

```

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl apply -f deployment.yaml
deployment.apps/frontend-deploy created

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
frontend-deploy  3/3     3            3          10s

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
frontend-deploy-566f4b78d6-fp9wm  1/1     Running   0          15s
frontend-deploy-566f4b78d6-lr9qj  1/1     Running   0          15s
frontend-deploy-566f4b78d6-prxs4  1/1     Running   0          15s
frontend-pod   1/1     Running   0          107s
frontend-rs-rb4tq  1/1     Running   0          66s
frontend-rs-vcnj8  1/1     Running   0          66s

```

service.yml

- Exposes application externally
- Routes traffic to pods
-

```

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl apply -f service.yaml
service/frontend-service created

npci@ASUS MINGW64 ~/OneDrive/Desktop/subbarayudu/AI-ML-ASSIGNMENTS/ASSIGNMENT-3/k8s (main)
• $ kubectl get svc
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
frontend-service  NodePort      10.96.121.12  <none>        80:30080/TCP     3s
kubernetes      ClusterIP     10.96.0.1     <none>        443/TCP          41d

```

V1 Running:

Browser showing:

ASSIGNMENT-3-V1
COMPLETED...

ASSIGNMENT-3-V1

COMPLETED...

By Subbarayudu Satti...

Update Deployment to v2:

Run image update command:

```
$ kubectl set image deployment/frontend-deploy frontend=subbarayudusatti/k8s-app:v2
```

This command updates deployment image from:

- v1 → v2

Monitor Rolling Update

Watch update process:

```
kubectl get pods -w
```

During update:

Old Pods terminate gradually

New Pods create gradually

Example process:

Old Pod → Terminating

New Pod → Creating

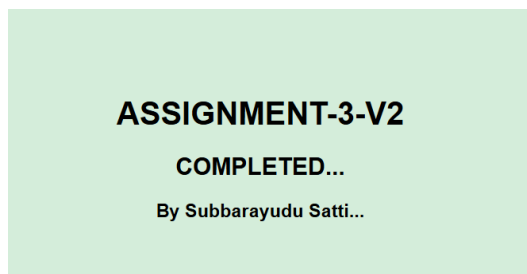
New Pod → Running

v2 Running

Browser showing:

ASSIGNMENT-3-V2

COMPLETED...



Conclusion:

This assignment successfully showcased the process of deploying and managing an application using Kubernetes in a practical environment.

The frontend application was first containerized with Docker and then deployed to a Kubernetes cluster using core Kubernetes components such as Pods, ReplicaSets, Deployments, and Services. To ensure the application remained stable and reliable, health checks were configured using Liveness, Readiness, and Startup Probes.

Initially, Version 1 of the application was deployed, and later it was upgraded to Version 2 through the Rolling Update strategy. During this process, Kubernetes smoothly replaced the old Pods with new ones without causing any downtime, ensuring uninterrupted application availability.

Through this assignment, valuable hands-on experience was gained in Docker image creation, version management, Kubernetes resource handling, service exposure, rolling updates, and container health monitoring.

Overall, the assignment provided a clear understanding of the Kubernetes deployment lifecycle and demonstrated how applications can be efficiently managed and updated in a containerized environment.